PRACTICAL DEVELOPER HANDBOOK

Git & GitHub

A beginner-friendly guide to owning your project history: commands, daily workflow, branches, collaboration, conflicts, and safe recovery.

Prepared for: ScamShield development

Environment: Windows, PowerShell, Android Studio

Edition: September 2026

Contents

1. The mental model
2. Vocabulary
3. Setup and authentication
4. Cloning and pulling repositories
5. The daily workflow
6. Commands explained
7. Collaboration and pull requests
8. Conflicts and recovery
9. Android Studio equivalents
10. Practice exercise
11. Cheat sheet

> **The habit that prevents most problems:** run `git status` before and after every important operation. Read Git's response carefully.

1. The mental model

Git tracks versions on your computer. **GitHub** hosts a remote Git repository and adds sharing, permissions, reviews, issues, and pull requests.

```
Working files → Staging area → Local commits → GitHub
   edit        git add       git commit       git push
```

- **Working files:** files being edited.
- **Staging area:** changes selected for the next commit.
- **Local repository:** history in your computer's hidden `.git` directory.
- **Remote repository:** shared history, normally on GitHub.

A commit is local until you push it. Git can work without internet; GitHub synchronization cannot.

2. Essential vocabulary

Term	Meaning
Repository	A project plus its complete Git history.
Commit	A saved snapshot with an ID, author, date, and message.
Branch	An isolated line of development.
`main` / `master`	The usual default branch. ScamShield currently uses `master`.
Remote	A named connection to another repository URL.
`origin`	The conventional name for the repository you cloned.
Clone	Download a repository for the first time, including history.
Fetch	Download remote history without changing current files.
Pull	Fetch, then integrate remote changes.
Push	Upload local commits.
Merge	Combine two branch histories.
Pull request	A GitHub proposal to review and merge branches. It is not `git pull`.
Fork	Your GitHub copy of someone else's repository.

3. One-time setup and authentication

Set the author name and email stored inside your commits:

```
git config --global user.name "Your Real Name"
git config --global user.email "your-github-email@example.com"
git config --global --list
```

`--global` applies to repositories on this computer. Omit it to set an identity only for the current repository.

For HTTPS remotes, sign in through Android Studio's browser prompt or Git Credential Manager. GitHub does not accept a normal account password for Git operations; use a personal access token if a terminal password prompt requires a credential.

> **Never commit secrets:** passwords, API keys, tokens, production signing keys, service-account files, or sensitive `.env` files. Android's `local.properties` normally stays untracked. Deleting a leaked secret later does not remove it from older history.

4. Getting a remote repository

Not on your computer yet: clone

```
cd C:\Users\Administrator\AndroidStudioProjects
git clone https://github.com/OWNER/REPOSITORY.git
cd REPOSITORY
git status
git remote -v
```

`clone` creates a directory, downloads files and history, adds a remote named `origin`, and checks out the default branch.

Already on your computer: fetch or pull

```
git status
git fetch origin
git pull --ff-only
```

`fetch` lets you inspect remote work safely. `pull` downloads and integrates it. `--ff-only` refuses to create an unexpected merge commit.

> **Before pulling:** make sure `git status` is clean. Commit meaningful changes or stash unfinished changes first.

Permission cases

- **Collaborator:** accept the invitation, clone the original repo, then push a feature branch.
- **No write permission:** fork the repository, clone your fork, then open a pull request back to the original.
- **Private repository:** you need permission and valid authentication even to clone it.

5. The safe daily workflow

Memorize this routine. Replace `main` with `master` for ScamShield or any repository using that default.

1 — Update your stable branch

```
git switch main
git pull --ff-only origin main
```

2 — Create a task branch

```
git switch -c feature/descriptive-name
```

Examples: `feature/dark-mode`, `fix/history-crash`, `docs/setup-guide`.

3 — Edit, test, and inspect

```
git status
git diff
```

4 — Stage deliberately

```
git add app/src/main/res/values/strings.xml
git diff --staged
git status
```

`git add -A` stages every new, modified, and deleted file. Use it only after reviewing status.

5 — Commit a focused result

```
git commit -m "Remove user profile fields"
```

Good messages complete "This commit will…" Examples: "Fix empty history crash" or "Add Swahili settings labels." Avoid "changes," "update," and "final."

6 — Push your branch

```
git push -u origin feature/descriptive-name
```

`-u` remembers the tracking relationship. Later, `git push` is sufficient.

7 — Open a pull request

1. Choose your feature branch as the source.
2. Choose `main` / `master` as the destination.

3. Explain what changed, why, and how you tested it.
4. Review the changed files yourself.
5. Merge after review and automated checks pass.

> Small, focused commits are easier to review, test, understand, and undo.

6. Commands explained

Command	Result	Changes files?
`git status`	Branch and file states	No
`git diff`	Unstaged line changes	No
`git diff --staged`	Exact next-commit content	No
`git log --oneline`	Compact commit history	No
`git remote -v`	Remote names and URLs	No
`git add FILE`	Select changes for commit	Staging only
`git commit -m "..."`	Create local snapshot	Creates history
`git fetch origin`	Download remote history	Not working files
`git pull --ff-only`	Download and fast-forward	Yes
`git push`	Upload local commits	Changes remote
`git switch BRANCH`	Change active branch	Yes
`git switch -c BRANCH`	Create and enter branch	Branch metadata

Read a push command word by word

```
git push -u origin feature/remove-profile
```

- `git` : run Git.
- `push` : upload commits.
- `-u` : remember tracking.
- `origin` : destination nickname.
- `feature/remove-profile` : branch to publish.

7. Collaboration, branches, and forks

Direct collaborator

```
git clone https://github.com/TEAM/PROJECT.git
cd PROJECT
git switch -c feature/my-change
# edit, test, add, commit
git push -u origin feature/my-change
# open a pull request on GitHub
```

Fork workflow

Use `origin` for your fork and `upstream` for the original:

```
git clone https://github.com/YOU/PROJECT.git
cd PROJECT
git remote add upstream https://github.com/ORIGINAL/PROJECT.git
git remote -v

git fetch upstream
git switch main
git merge --ff-only upstream/main
git push origin main
```

Create a feature branch, push it to your fork, then open a pull request from your fork/feature branch into the original/main branch.

Update a feature branch

```
git fetch origin
git merge origin/main
```

Some teams prefer rebase. Follow the team convention. Do not rebase a shared branch without coordination because rebase rewrites commit identities.

8. Conflicts and safe recovery

A conflict means Git cannot decide how to combine edits, commonly because two people changed the same lines.

```
<<<<<<< HEAD
Your current version
=======
The incoming version
>>>>>>> origin/main
```

1. Run `git status`.

2. Understand both versions.
3. Edit the correct final result and remove markers.
4. Build and test.
5. Run `git add FILE` for resolved files.
6. Complete the merge with `git commit` if requested.

Abandon an unfinished merge with:

```
git merge --abort
```

> Do not blindly choose “Accept Yours” or “Accept Theirs.” The correct result may combine both.

Undo guide

Situation	Safe command
Staged the wrong file	`git restore --staged PATH` — preserves edits.
Pause unfinished work	`git stash push -m "unfinished work"`, later `git stash pop`.
Undo a published commit	`git revert COMMIT_HASH` — adds a reverse commit.
Inspect a commit	`git show COMMIT_HASH`.
Find commit IDs	`git log --oneline --decorate -15`.

> **High risk:** `git reset --hard` can destroy uncommitted work. `git push --force` can overwrite shared history. Never use either as a general fix.

Before a complex operation, name the current commit as a safety point:

```
git branch backup/before-merge
```

This does not preserve uncommitted edits; commit or stash those separately.

9. Android Studio equivalents

Goal	Android Studio	Command
Inspect	Commit tool window	`git status` / `git diff`
Commit	`Ctrl+K`	`git commit`
Push	`Ctrl+Shift+K`	`git push`
Fetch	Git → Fetch	`git fetch`
Pull	Git → Pull	`git pull --ff-only`
Branch	Bottom-right branch widget	`git switch`
History	Git tool window → Log	`git log`
Conflicts	Git → Resolve Conflicts	Edit, add, commit

Use Android Studio's visual diffs, but learn the terminal vocabulary. The interface becomes predictable when you understand the underlying operation.

10. Practice exercise

Practice in a disposable repository named `git-practice`, not an important project.

1. Create the empty repository on GitHub.
2. Clone it.
3. Create a README with one sentence.
4. Inspect, stage, and commit it.
5. Push the default branch.
6. Create `feature/add-notes`.
7. Edit and commit the README.
8. Push the branch and open a pull request.
9. Merge it on GitHub.
10. Switch locally to the default branch and pull.

```
git clone https://github.com/YOU/git-practice.git
cd git-practice
# create README.md, then:
git status
git add README.md
git diff --staged
git commit -m "Add project README"
git push

git switch -c feature/add-notes
# edit README.md, then:
git add README.md
git commit -m "Add learning notes"
git push -u origin feature/add-notes
# merge the pull request on GitHub
git switch main
git pull --ff-only
git log --oneline --graph --decorate --all
```

11. Daily cheat sheet

Inspect without changing anything

```
git status
git diff
git diff --staged
git branch --show-current
git remote -v
git log --oneline --graph --decorate --all -15
```

Start, save, and publish a task

```
git switch main
git pull --ff-only origin main
git switch -c feature/task-name
# edit and test
git status
git add path/to/file
git diff --staged
git commit -m "Describe the result"
git push -u origin feature/task-name
```

Common errors

Problem	Meaning / response
`not a git repository`	You are in the wrong directory. Use `cd` to enter the project.
`nothing to commit`	No changes are staged; inspect status and directory.
Push rejected: non-fast-forward	Remote has newer commits. Fetch, inspect, integrate, test, and push. Do not jump to force push.
`origin already exists`	Inspect `git remote -v`; change it only if wrong.
Authentication failed	Check permission and sign in with browser/token/credential manager.
Merge conflict	Inspect status, resolve each file, test, stage, and finish the merge.

Before a commit

1. Correct branch?
2. Project tested?
3. Only intentional files?
4. Reviewed `git diff --staged`?
5. No secrets or generated files?

6. Clear commit message?

Before a pull

1. Working tree clean?
2. Correct branch?
3. Unusual remote changes inspected?

Before a push

1. Commits tested?
2. Correct branch?
3. Correct remote and destination shown?

> **Final rule:** make small commits, use feature branches, inspect before acting, and collaborate through pull requests. When uncertain, preserve the current state and investigate instead of guessing with destructive commands.

Official references: docs.github.com (cloning, remote changes, branches, pull requests, remotes); git-scm.com/docs; developer.android.com/studio/projects/version-control.